

Relatório – Disciplina SD212
Atividade 29-05-2026
Análise de Controlador de Interrupções

01. Objetivo Geral do Repositório

O repositório disponibiliza uma implementação didática em Verilog de um controlador de interrupções com 8 entradas. O projeto demonstra como periféricos sinalizam eventos ao processador sem que o software precise ficar verificando continuamente cada dispositivo. Suporta dois modos: polling (varredura sequencial) e custom priority (ordem configurável), e inclui testbench completo para verificação funcional.

02. Quantas entradas de interrupção o controlador suporta?

O controlador suporta 8 fontes de interrupção independentes, representadas pelo barramento de 8 bits `intr_rq[7:0]`. Cada bit corresponde a uma fonte de interrupção distinta, podendo estar ativo (1) ou inativo (0) simultaneamente.

```
module INTR_CNTRL (  
    input  wire      clk_in,    // Clock  
    input  wire      rst_in,    // Reset  
    input  wire [7:0] intr_rq,   // Interrupt request  
    inout  wire [7:0] intr_bus,  // Bidirectional data bus  
    input  wire      intr_in,    // Ack from processor  
    output wire      intr_out,   // Interrupt to processor  
    output wire      bus_oe     // Bus output enable  
);
```

03. Quais são os principais arquivos Verilog presentes na pasta src?

O arquivo principal da pasta **src** é:

- `intrCntrl.v` — módulo principal do controlador de interrupções (`INTR_CNTRL`). Implementa a máquina de estados finita (FSM) com 16 estados, os registradores internos, a lógica de prioridade e o barramento bidirecional.

04. Quais arquivos estão presentes na pasta testbench?

A pasta `testbench` contém:

- `intrCntrl_tb.v` — testbench do controlador. Instancia o DUT (Device Under Test), gera clock, reset, estímulos de interrupção e simula as respostas do processador nos dois modos de operação.

05. Quais sinais representam as fontes de interrupção?

Os Sinais de Fonte de interrupção:

Sinal	Largura	Descrição
<code>intr_rq[7:0]</code>	8 bits	Fontes de interrupção externas (periféricos)
<code>intr_bus[7:0]</code>	8 bits	Barramento bidirecional: comandos e IDs de interrupção
<code>intr_in</code>	1 bit	Acknowledgement do processador (ativo em nível baixo)

06. Qual sinal indica ao processador que existe uma interrupção ativa?

O sinal `intr_out` (1 bit) vai a nível alto quando o controlador detecta uma fonte ativa, sinalizando ao processador que há um evento pendente. O sinal `bus_oe` indica quando o controlador está conduzindo o barramento bidirecional

```
413 // Interrupt output. It's the same as the intrOut_reg but done like this for clarity.
414 assign intr_out = intrOut_reg;
415 // Bus is bidirectional. oe (output enable) decides if the controller is driving it
416 // or is expecting input on it.
417 assign intr_bus = (oe_reg) ? intrBus_reg : 8'bzzzzzzzz;
418 // Used by the testbench. This signal tells the processor if the bus is being actively
419 // driven by the controller.
420 assign bus_oe = oe_reg;
```

07. Como funciona o modo polling descrito no README?

No modo polling, o processador informa o controlador enviando o padrão `xxxx_xx01` no barramento `intr_bus` (bits [1:0] = 01). A partir desse ponto, o controlador percorre ciclicamente todas as 8 fontes de interrupção, verificando uma por ciclo de clock, do índice 0 ao 7. Não há prioridade — a primeira fonte ativa encontrada é atendida.

Sequência de operação:

- O controlador detecta `intr_rq[N]` ativo e eleva `intr_out`.
- O processador reconhece com transição High→Low→High em `intr_in`.
- O controlador coloca no barramento o código `01011_NNN` (onde NNN é o ID da fonte).
- O processador confirma o endereço com novo pulso em `intr_in`.
- Após executar a rotina de serviço (ISR), o processador envia `10100_NNN` no barramento com pulso em `intr_in`.
- O controlador volta a varrer as fontes a partir do próximo índice.

```
1 // Trecho - S_StartPolling
2 S_StartPolling: begin
3   if (intr_rq[intrIndex_reg]) begin
4     intrOut_next = 1'b1; // Sinaliza interrupção
5     state_next = S_TxIntInfoPolling;
6   end else begin
7     intrIndex_next = intrIndex_reg + 1; // Próxima fonte
8   end
9 end
10
11 // Trecho - S_TxIntInfoPolling: envia ID ao processador
12 if (~intr_in) begin
13   intrBus_next = {5'b01011, intrIndex_reg}; // Código 01011 + ID
14   oe_next = 1'b1;
15   state_next = S_AckTxInfoRxPolling;
16 end
17
18 end
```

08. Como funciona o modo custom priority?

Neste modo, o processador envia o padrão `xxxyyy10` no barramento em 4 ciclos consecutivos, definindo a ordem de prioridade de todas as 8 fontes. Nos bits [7:5] e [4:2] de cada ciclo são codificados, respectivamente, os IDs de dois periféricos em ordem de prioridade decrescente:

- Ciclo 1: prioridades 0 (mais alta) e 1

- Ciclo 2: prioridades 2 e 3
- Ciclo 3: prioridades 4 e 5
- Ciclo 4: prioridades 6 e 7 (mais baixa)

Após a inicialização, o controlador verifica as fontes na ordem definida pela tabela de prioridade interna, não em sequência numérica. Os códigos de condição mudam: o controlador envia 10011_NNN e o processador responde com 01100_NNN.

09. Qual é a função do sinal de acknowledgement vindo do processador?

O sinal `intr_in` é um canal de confirmação de mão dupla entre processador e controlador, ativo em nível baixo (pulso High→Low→High). Ele é utilizado em três momentos distintos no protocolo:

- 1ª confirmação: o processador reconhece que recebeu o sinal `intr_out` (interrupção ativa).
- 2ª confirmação: o processador confirma que recebeu o endereço/ID da fonte no barramento.
- 3ª confirmação: o processador sinaliza que a ISR foi concluída (junto com código de condição no barramento).

Caso os códigos de condição recebidos na última confirmação não coincidam com o ID esperado, o controlador vai para o estado de reset, indicando erro de protocolo

10. O que acontece após o reset do controlador?

Quando o sinal `rst_in` é ativado (nível alto), o controlador:

- Vai para o estado `S_Reset` (4'b0000).
- Zera todos os registradores internos: `cmdMode`, `cmdCycle`, `intrIndex`, `intrPtr`.
- Limpa a tabela de prioridade `prior_table[0..7]`.
- Desabilita o output enable do barramento (`bus_oe = 0`) e tristata `intr_bus`.
- Em seguida, transita automaticamente para `S_GetCommands`, onde aguarda o comando de modo via barramento.

```

1 // Trecho - bloco de reset síncrono
2 if (rst_in) begin
3     state_reg      <= S_Reset;      // Estado 0000
4     cmdMode_reg    <= 2'b00;
5     oe_reg         <= 1'b0;        // Barramento tristestado
6     intrOut_reg    <= 1'b0;
7     intrIndex_reg  <= 3'b000;
8     for (i=0;i<8;i=i+1) prior_table_reg[i] <= 3'b000;
9 end

```

11. Como o testbench estimula o controlador de interrupções?

O testbench simula o papel do processador, gerando estímulos ordenados:

- Aplica reset por 20 ns e depois libera.
- Envia o comando de modo (01 ou sequência de 10) pelo barramento `intr_bus_in`.
- Ativa fontes de interrupção via `intr_rq`.
- Aguarda `intr_out` subir (`wait(intr_out)`).
- Gera pulsos em `intr_in` para simular as três confirmações do protocolo.
- Lê o barramento `intr_bus_out` para verificar o ID e código retornado pelo controlador.

- Envia o código de encerramento da ISR e passa para a próxima interrupção.

Asserções explícitas verificam se o barramento contém os valores esperados, encerrando a simulação com erro caso haja divergência.

12. O testbench cobre quais situações principais?

O testbench cobre dois cenários principais:

Modo Polling: 8 interrupções servidas em sequência. Na 5ª interrupção as fontes ativas mudam (`intr_rq = 8'b0101_0101`), testando a adaptação do controlador a novas solicitações.

Modo Custom Priority: 10 interrupções servidas na ordem $5 \rightarrow 3 \rightarrow 7 \rightarrow 0 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 5 \rightarrow 6 \rightarrow 1$. Fontes adicionais são ativadas durante a sequência (bit 3 na 5ª iteração e bit 5 na 7ª), verificando que prioridades dinâmicas são respeitadas.

13. Qual a diferença entre polling e interrupção no contexto da aula?

Aspecto	Polling (software)	Interrupção (hardware)
Iniciativa	Processador verifica periodicamente	Periférico sinaliza quando pronto
CPU ocupada	Sim — loop de verificação	Não — executa outra tarefa
Latência	Variável (depende do ciclo)	Determinística e menor
Complexidade HW	Baixa	Requer controlador dedicado

No repositório, o 'modo polling' do controlador ainda usa interrupção para avisar o processador — a diferença é somente na forma como o controlador varre as fontes internamente (sequencial vs. por prioridade).

14. Por que um controlador de interrupções é útil em um SoC?

Em um SoC, dezenas ou centenas de periféricos (UART, SPI, timers, GPIOs, etc.) podem gerar eventos simultaneamente. Um controlador de interrupções centraliza o gerenciamento desses eventos, oferecendo:

- Arbitragem de prioridade: garante que eventos críticos sejam atendidos antes de eventos de baixa criticidade.
- Identificação automática da fonte: o processador não precisa consultar cada periférico para descobrir quem gerou a interrupção.
- Mascaramento seletivo: fontes podem ser habilitadas ou desabilitadas individualmente sem alterar o hardware.
- Redução de latência: o núcleo do processador recebe um único sinal (`intr_out`) e o ID já pronto, simplificando o despacho da ISR.
- Economia de pinos: múltiplos periféricos compartilham um único canal de interrupção ao processador.

15. O que seria necessário acrescentar para aproximar esse controlador de um sistema embarcado real?

- Registro de máscara (IMR): permitir habilitar/desabilitar individualmente cada fonte de interrupção por software.
- Registro de status (ISR): guardar quais interrupções estão pendentes, mesmo que já tenham sido reconhecidas.

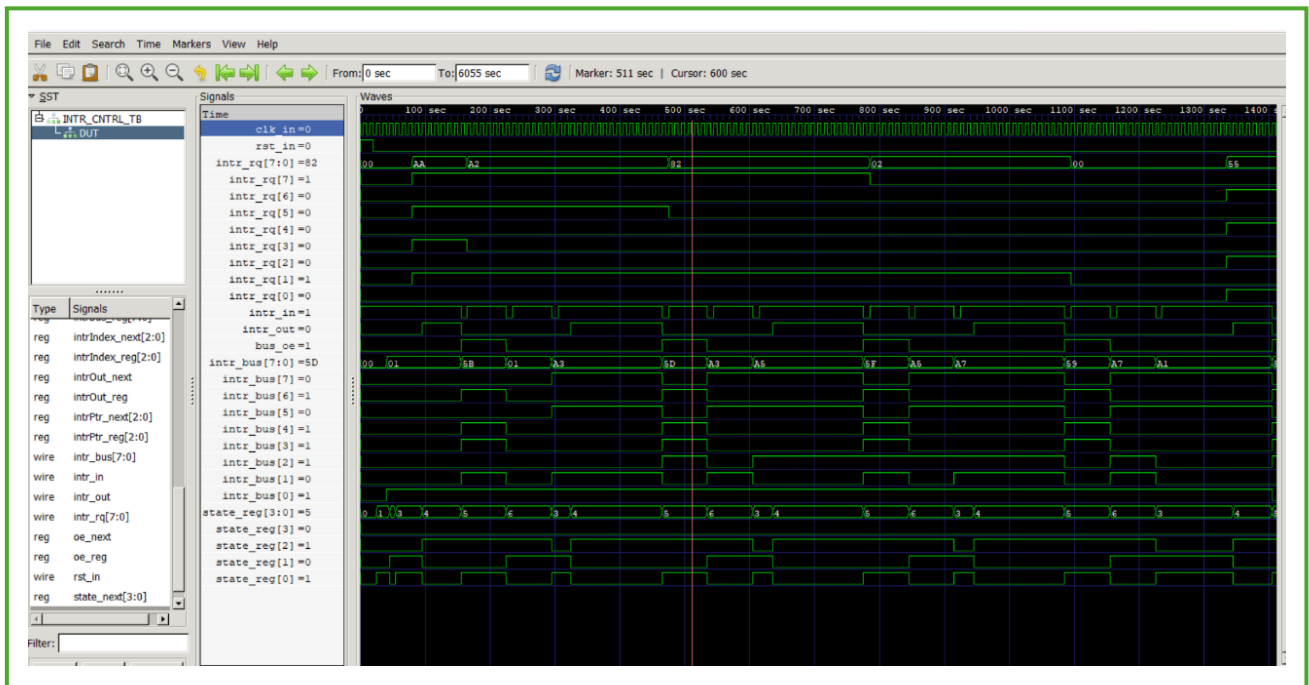
- Prioridades reconfigurável em tempo de execução: sem necessidade de reset para alterar a tabela.
- Suporte a interrupções aninhadas (nested): permitir que uma ISR de alta prioridade interrompa uma de baixa prioridade já em execução.
- Detecção de borda vs. nível: filtrar fontes sensíveis a borda de subida/descida ao invés de apenas nível alto.

Verificação de duplicidade na tabela de prioridade: o próprio README aponta esta lacuna — duas entradas com o mesmo ID podem causar comportamento indefinido

16. Simulação – Formas de Onda

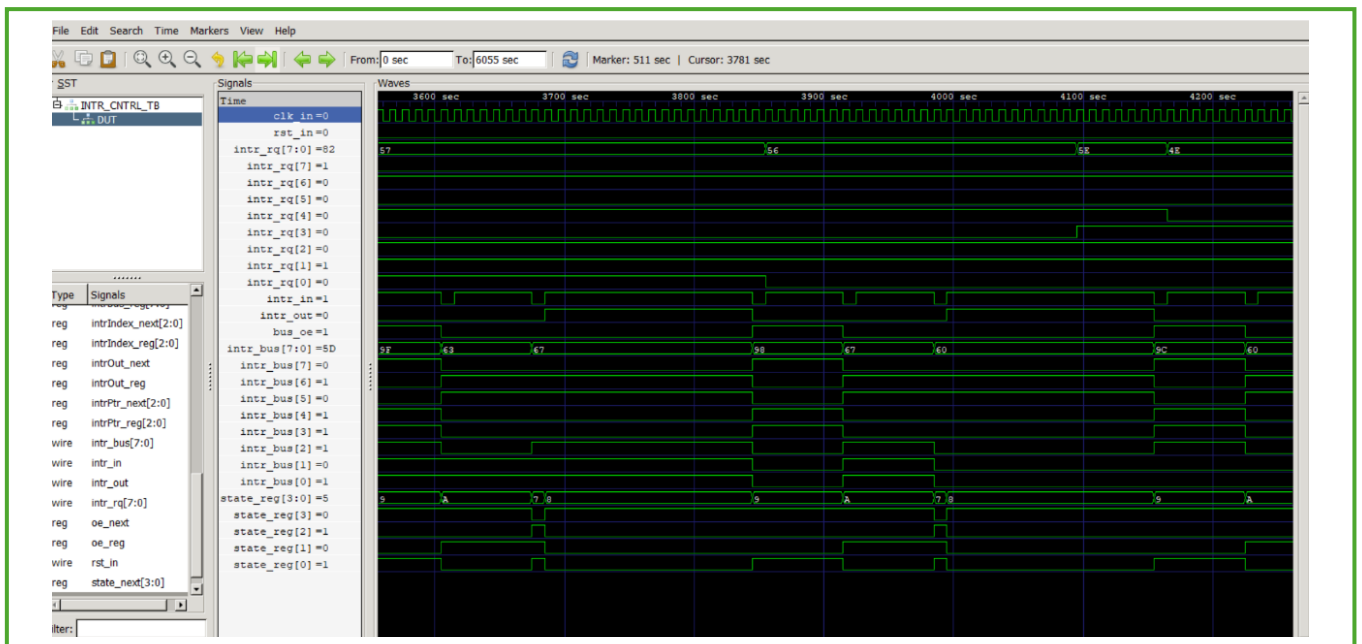
A simulação foi executada com Icarus Verilog e visualizada no GTKWave. Os sinais monitorados foram: clk_in, rst_in, intr_rq[7:0], intr_out, intr_in, bus_oe, intr_bus[7:0] e state_reg[3:0].

16.1 Fase Polling(0 – 1400 sec)



Sinal	Valor observado	Significado
intr_rq[7:0]	AA → 55	Fontes 1,3,5,7 ativas; mudam para 0,2,4,6 na 5ª iteração
intr_out	Pulsos altos	Controlador sinalizando interrupção a cada fonte detectada
intr_bus	5B, 5D, 5F...	Código 01011_NNN — polling; NNN = ID da fonte
state_reg	3→4→5→6	StartPolling→TxInfo→AckTxInfo→AckISRDone
intr_in	3 pulsos baixos	Os três acknowledgements do processador por interrupção
bus_oe	Alto entre acks	Controlador conduzindo o barramento com o ID

16.2 – Fase Priority (~3600 – 4200 sec)



Sinal	Valor observado	Significado
intr_bus	9F,63,67,98...	Código 10011_NNN — priority; NNN = ID na ordem de prioridade
state_reg	7→8→9→A	StartPriority→TxInfo→AckTxInfo→AckISRDone
intr_rq	Decrescente	Fontes desativadas conforme atendidas na ordem 5→3→7→0→4...
intr_in	3 pulsos baixos	Mesmo protocolo de 3 acks mantido no modo priority

Resultado Final do terminal

```

===== Beginning Polling =====
Servicing Interrupt Number 1 / 8 ... Proper address on the bus
Servicing Interrupt Number 2 / 8 ... Proper address on the bus
... (8 interrupções servidas com sucesso)

===== Beginning Priority =====
Proper Order - 5 -> 3 -> 7 -> 0 -> 4 -> 3 -> 2 -> 5 -> 6 -> 1
Servicing Interrupt Number 1 / 10 ... Proper address on the bus
... (10 interrupções servidas com sucesso)

===== All tests completed successfully =====

```